

AULA 4

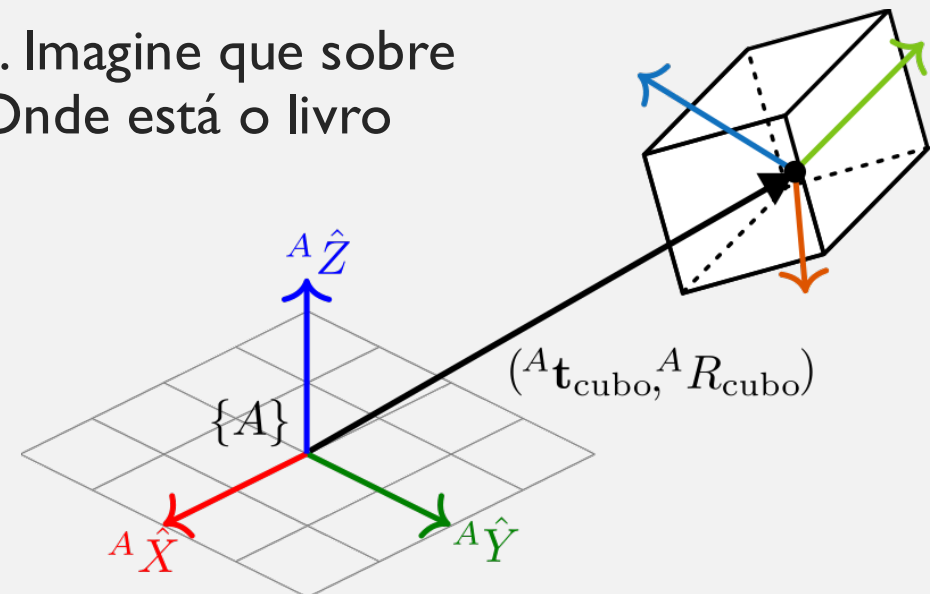
Sistemas de coordenadas, TF2 e Introdução ao RViz

NA AULA DE HOJE...

- Introdução a sistemas de coordenadas
 - Operações entre sistemas de coordenadas
 - Transformações espaciais
- Introdução ao pacote tf2
- Demonstrações no RVIZ

INTRODUÇÃO A SISTEMAS DE COORDENADAS

- Sempre que nós estivermos tratando do mundo 3D, temos que pensar em *sistemas de coordenadas* (também chamados de *frames*), e das relações entre eles. Por exemplo:
 - Imagine que tem um laptop no topo de uma mesa. Imagine que sobre essa mesa há também um livro, rotacionado 45° . Onde está o livro com relação a...
 - ...a tecla “esc” do laptop?
 - ...o pé esquerdo traseiro da mesa?
 - ...a sala?
 - Tudo isso depende dos nossos *frames*!



PONTOS E TRANSLAÇÃO

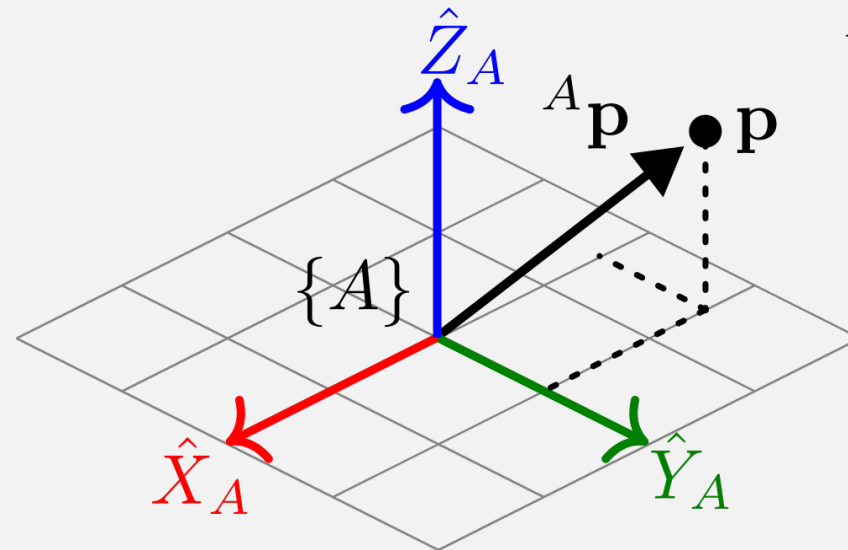
- Sempre que quisermos localizar um ponto, precisamos especificar com relação a *qual* referência ele está localizado
- Considere a figura ao lado, com o ponto $\mathbf{p}$
 - Ele pode ser expresso em coordenadas do sistema $\{A\}$

É comum usar a seguinte convenção de cores para representar sistemas de coordenadas

$\hat{X}$
Red

$\hat{Y}$
Green

$\hat{Z}$
Blue



$${}^A\mathbf{p} = \begin{bmatrix} {}^A p_x \\ {}^A p_y \\ {}^A p_z \end{bmatrix}$$

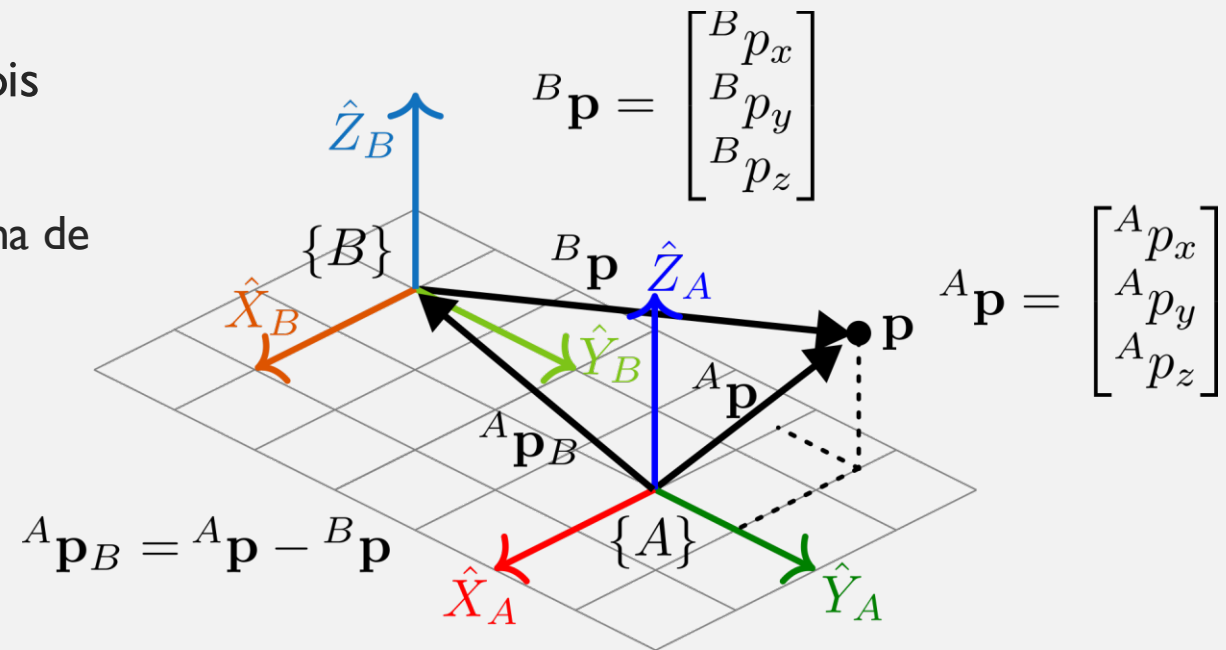


Os eixos *sempre* são perpendiculares entre si, e os eixos *sempre* seguem a regra da mão direita!
 $\hat{X} \times \hat{Y} = \hat{Z}$, onde $\times$ é o produto cruzado.

PONTOS E TRANSLAÇÃO

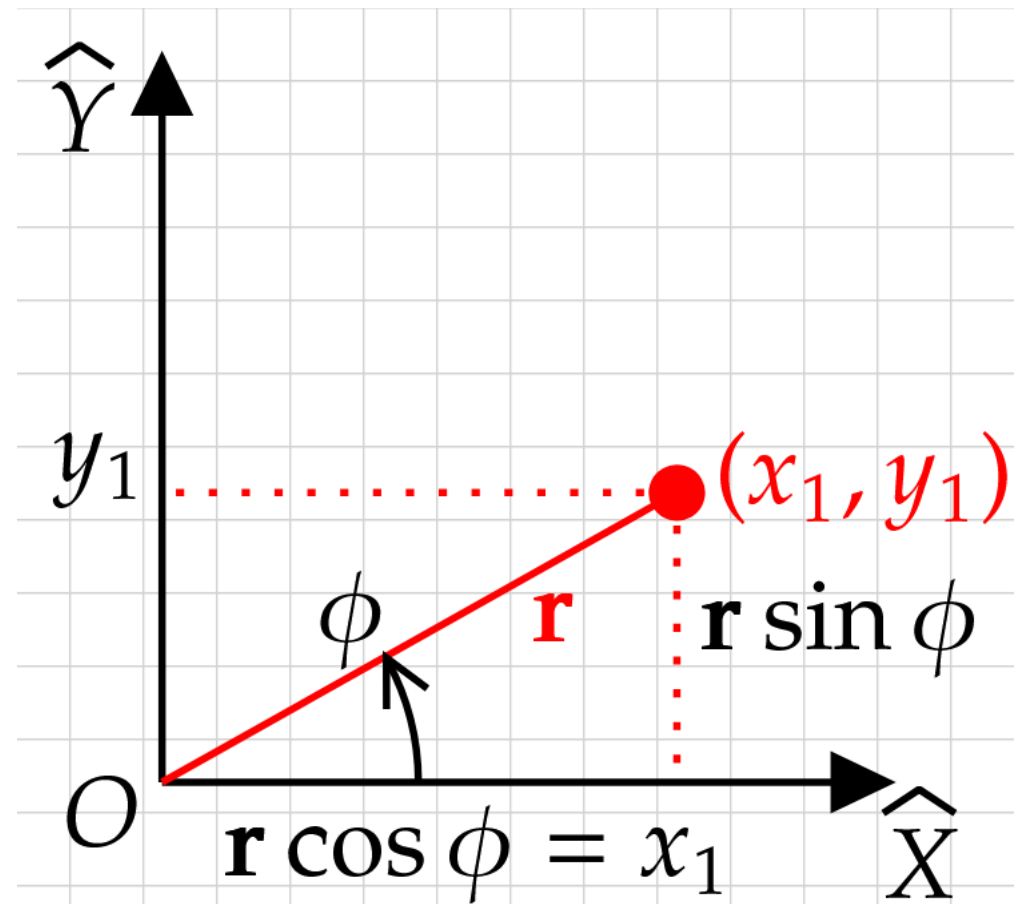
- ...mas o nosso ponto $\mathbf{p}$ também pode ser expresso com relação ao sistema $\{B\}$
- Como fica a transformação de coordenadas entre dois sistemas com a mesma orientação?
 - Denotemos ${}^A\mathbf{p}_B$ a origem de $\{B\}$ com relação ao sistema de coordenadas $\{A\}$
 - Pela figura: ${}^A\mathbf{p} = {}^A\mathbf{p}_B + {}^B\mathbf{p} = {}^A\mathbf{p} - \cancel{{}^B\mathbf{p}} + \cancel{{}^B\mathbf{p}}$
 - Evidentemente poderíamos fazer o inverso:

$${}^B\mathbf{p} = {}^B\mathbf{p}_A + {}^A\mathbf{p} = {}^B\mathbf{p} - \cancel{{}^A\mathbf{p}} + \cancel{{}^A\mathbf{p}}$$
 onde ${}^B\mathbf{p}_A = -{}^A\mathbf{p}_B$



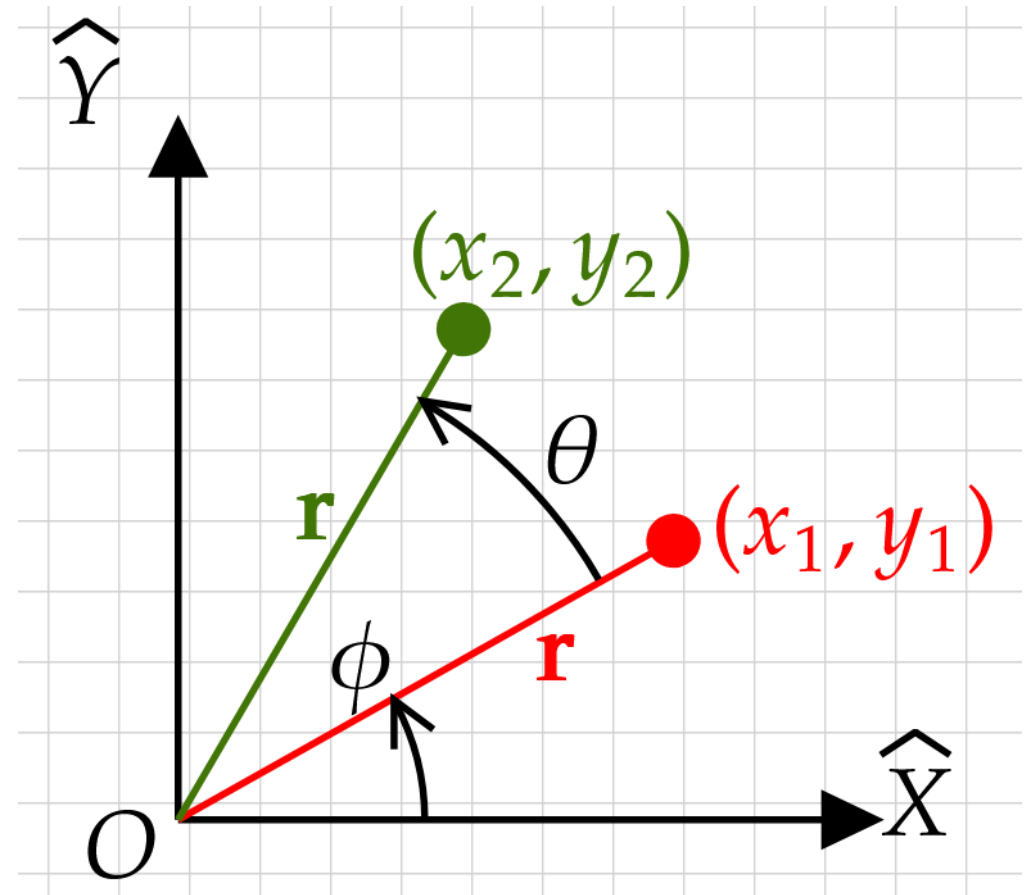
ROTAÇÕES NO PLANO

- Considere um ponto no plano, com coordenadas (x_1, y_1) . Ele está a uma distância $r = \sqrt{x_1^2 + y_1^2}$ da origem O .
- Em notação vetorial: $\begin{bmatrix} x_1 \\ y_1 \end{bmatrix} = \begin{bmatrix} r \cos \phi \\ r \sin \phi \end{bmatrix}$



ROTAÇÕES NO PLANO

- Então $\begin{bmatrix} x_1 \\ y_1 \end{bmatrix} = \begin{bmatrix} r \cos \phi \\ r \sin \phi \end{bmatrix}$
- O que acontece se rotacionarmos ele por θ graus em torno de O ?



ROTAÇÕES NO PLANO

- O que acontece se rotacionarmos ele por θ graus em torno de O ?

$$\begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = \begin{bmatrix} r \cos(\phi + \theta) \\ r \sin(\phi + \theta) \end{bmatrix} = \begin{bmatrix} r c_\phi c_\theta - r s_\phi s_\theta \\ r s_\phi c_\theta + r c_\phi s_\theta \end{bmatrix}$$

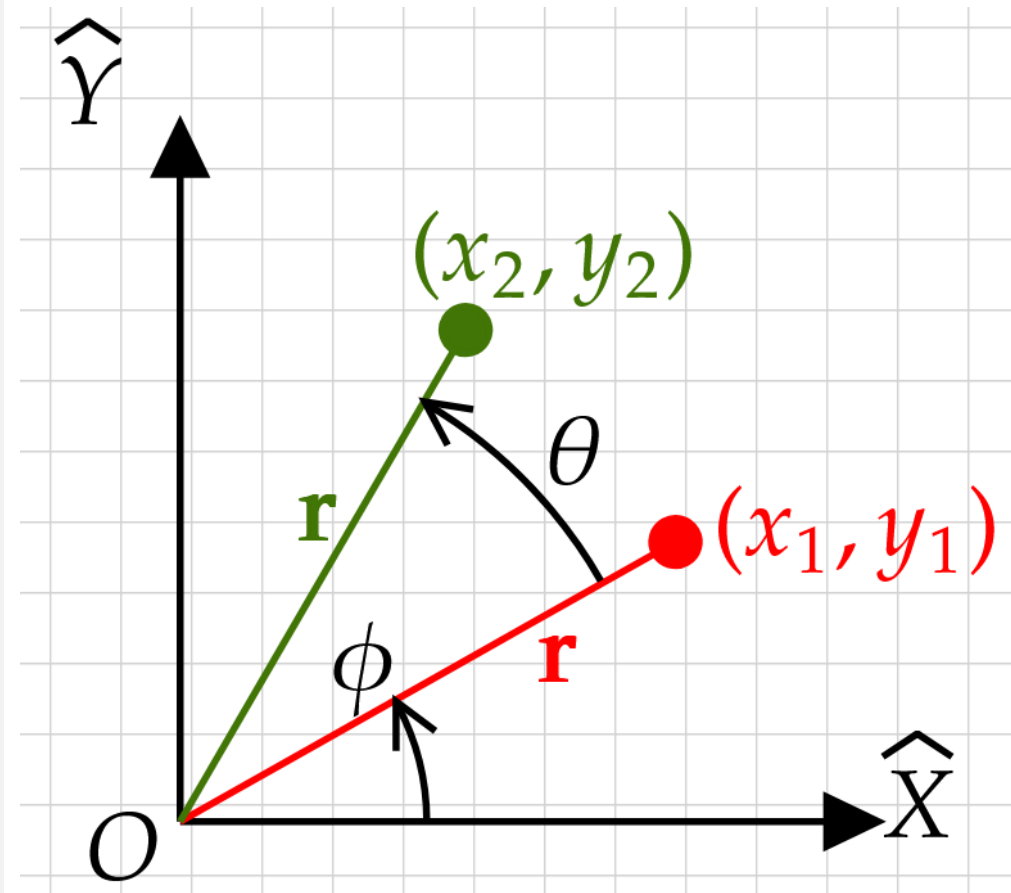
Substituindo $\begin{bmatrix} x_1 \\ y_1 \end{bmatrix} = \begin{bmatrix} r \cos \phi \\ r \sin \phi \end{bmatrix}$:

$$\begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = \begin{bmatrix} c_\theta & -s_\theta \\ s_\theta & c_\theta \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \end{bmatrix}$$

- Ou seja, é uma multiplicação matricial!

$$\mathbf{x}_2 = {}^1\mathbf{R}_2(\theta)\mathbf{x}_1$$

- Isso é uma rotação *ativa*; estamos mais interessados nas rotações *passivas*



*NOTA: Escrevemos $c_x = \cos x$, $s_x = \sin x$ para simplificar a notação.

ROTAÇÕES NO PLANO

- Rotação passiva: como escrever o ponto $\mathbf{P} = ({}^A x_P, {}^A y_P)$, descrito em $\{A\}$, com relação a $\{B\}$?
- Da figura, podemos ver que:

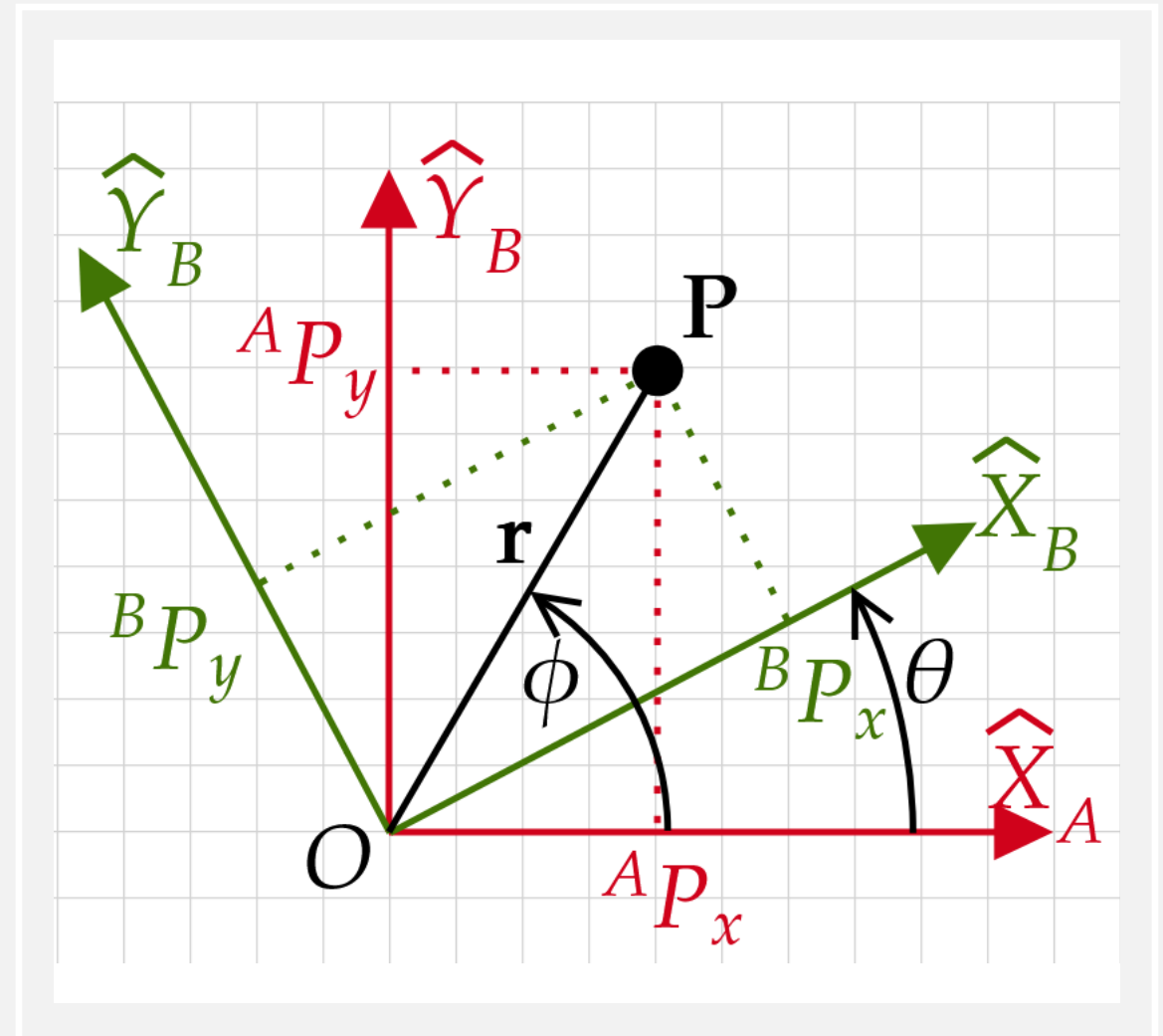
$$\bullet \quad {}^A P = \begin{bmatrix} {}^A P_x \\ {}^A P_y \end{bmatrix} = \begin{bmatrix} r c_\phi \\ r s_\phi \end{bmatrix} \quad (1)$$

$$\bullet \quad {}^B P = \begin{bmatrix} {}^B P_x \\ {}^B P_y \end{bmatrix} = \begin{bmatrix} r c_{\phi-\theta} \\ r s_{\phi-\theta} \end{bmatrix} = \begin{bmatrix} r c_\phi c_\theta + r s_\phi s_\theta \\ r s_\phi c_\theta - r c_\phi s_\theta \end{bmatrix} \quad (2)$$

$$\bullet \quad \text{Substituindo (1) em (2): } \begin{bmatrix} {}^B P_x \\ {}^B P_y \end{bmatrix} = \underbrace{\begin{bmatrix} c_\theta & s_\theta \\ -s_\theta & c_\theta \end{bmatrix}}_{{}^B R_A} \begin{bmatrix} {}^A P_x \\ {}^A P_y \end{bmatrix}$$

$$\bullet \quad \text{Logo: } \boxed{{}^B P = {}^B R_A(\theta) {}^A P} \quad \text{e} \quad \boxed{{}^A P = {}^B R_A^{-1}(\theta) {}^B P}$$

A matriz ${}^B R_A$ representa a orientação de $\{A\}$ em $\{B\}$; pontos com coordenadas em $\{A\}$ são transformados para sua representação em $\{B\}$



GENERALIZANDO PARA ROTAÇÕES TRIDIMENSIONAIS

- Considere um sistema de coordenadas tridimensional, com eixos $(\hat{X}, \hat{Y}, \hat{Z})$, sempre satisfazendo $\hat{Z} = \hat{X} \times \hat{Y}$

- A matriz de rotação de B para A é:

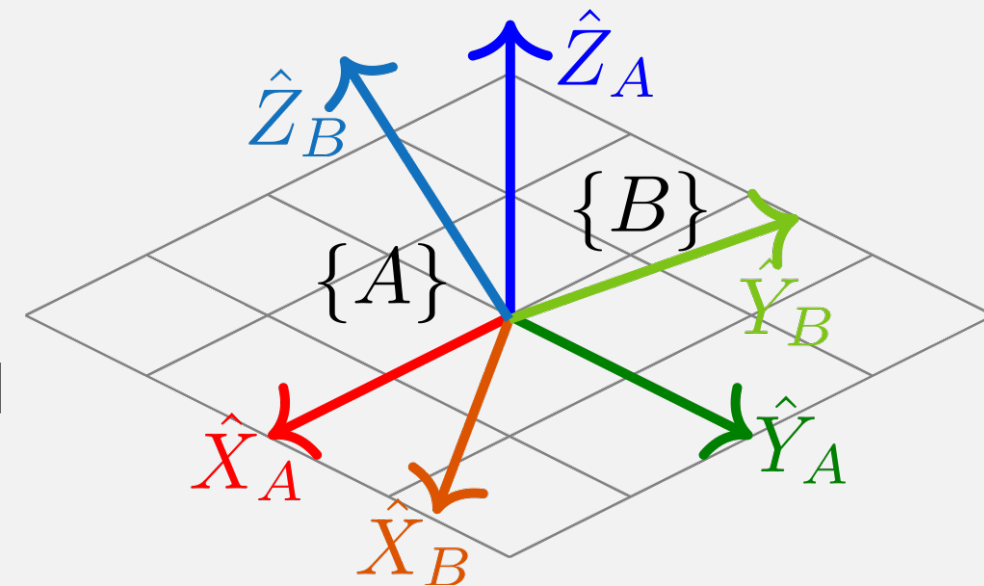
$${}^A R_B = \begin{bmatrix} \hat{X}_B \cdot \hat{X}_A & \hat{Y}_B \cdot \hat{X}_A & \hat{Z}_B \cdot \hat{X}_A \\ \hat{X}_B \cdot \hat{Y}_A & \hat{Y}_B \cdot \hat{Y}_A & \hat{Z}_B \cdot \hat{Y}_A \\ \hat{X}_B \cdot \hat{Z}_A & \hat{Y}_B \cdot \hat{Z}_A & \hat{Z}_B \cdot \hat{Z}_A \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} = [{}^A \hat{X}_B \quad {}^A \hat{Y}_B \quad {}^A \hat{Z}_B]$$

- Algumas propriedades relevantes:

- ${}^A R_B^{-1} = {}^B R_A^T \Leftrightarrow R^T R = R R^T = I$
- $\det R = 1 \Rightarrow$ (linhas, colunas) são ortonormais entre si
- Se R_1 e R_2 são matrizes de rotação, $R_3 = R_1 R_2$ também é
- **$R_1 R_2 \neq R_2 R_1$ (no caso geral)**
- $(R_1 R_2) R_3 = R_1 (R_2 R_3)$



**Rotações em 3D
(diferentemente do
2D) não são
comutativas!**

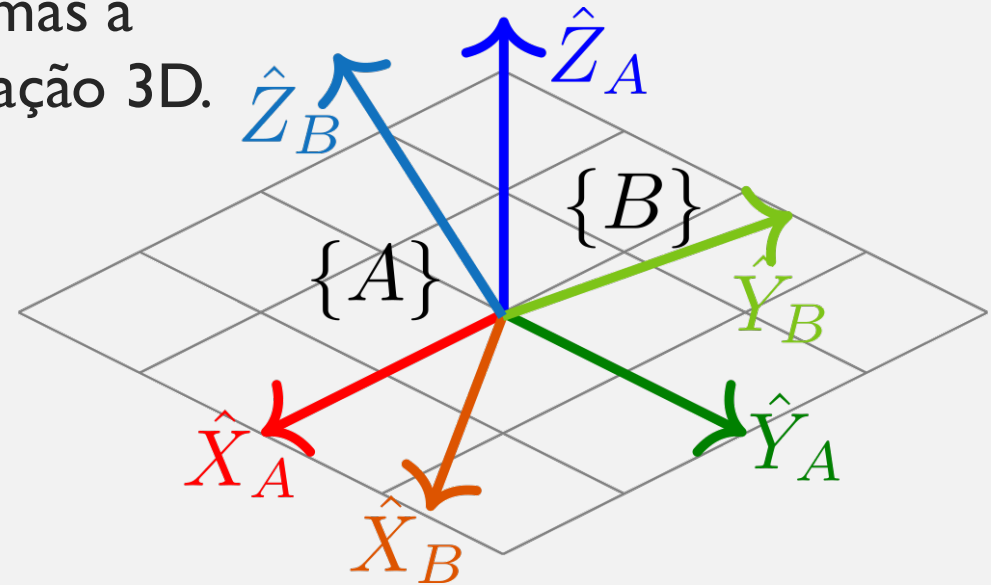


PARAMETRIZANDO ROTAÇÕES 3D

- Uma matriz de rotação é só *uma forma* (mas a mais específica) de parametrizar uma rotação 3D.

Outras parametrizações:

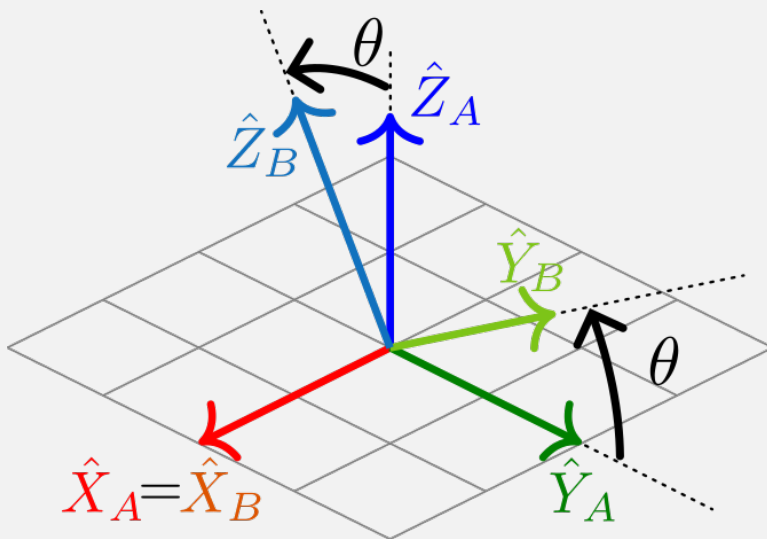
- Ângulos de Euler (Roll-Pitch-Yaw)
 - Eixo-ângulo
 - Quaternions
 - Coordenadas exponenciais
- Vamos passar aqui pelas mais relevantes...



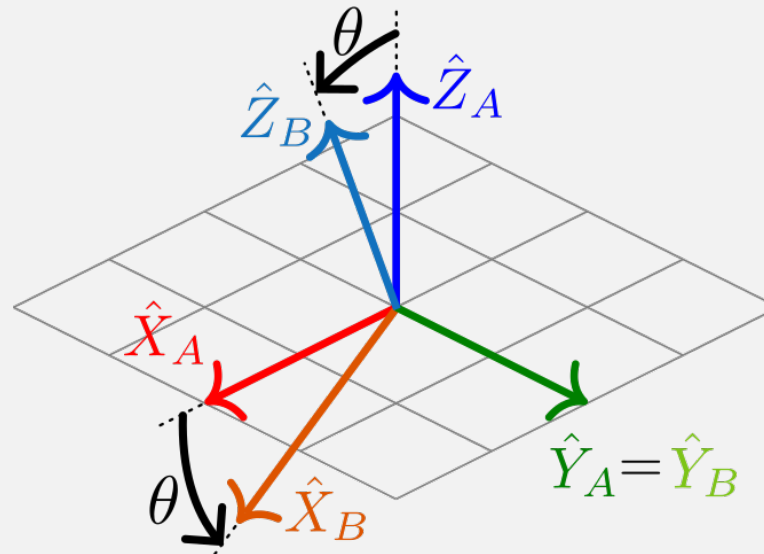
Note que todas essas parametrizações já foram implementadas (em várias) bibliotecas, e.g. [eigen](#) – mas é importante saber como usá-las, e o que a sua biblioteca está fazendo

ROTAÇÕES SOBRE EIXOS UNITÁRIOS

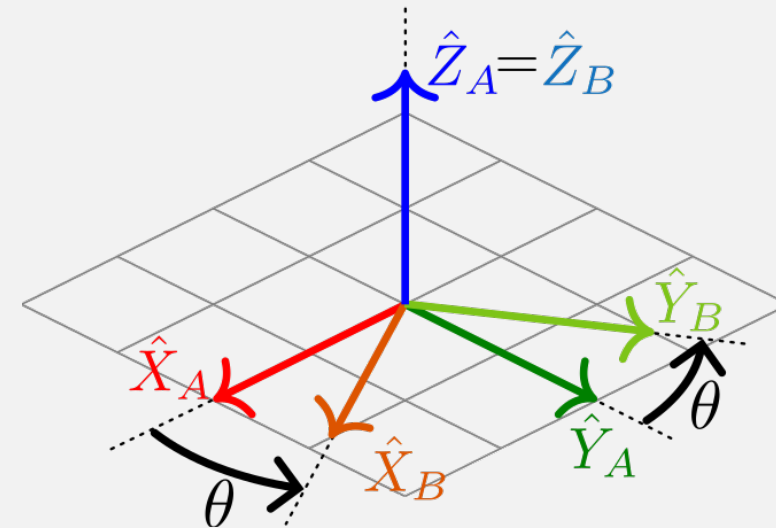
- As rotações elementares nos eixos $\hat{X}$, $\hat{Y}$, $\hat{Z}$ são frequentemente usadas:



$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c_\theta & -s_\theta \\ 0 & s_\theta & c_\theta \end{bmatrix}$$



$$R_y(\theta) = \begin{bmatrix} c_\theta & 0 & s_\theta \\ 0 & 1 & 0 \\ -s_\theta & 0 & c_\theta \end{bmatrix}$$

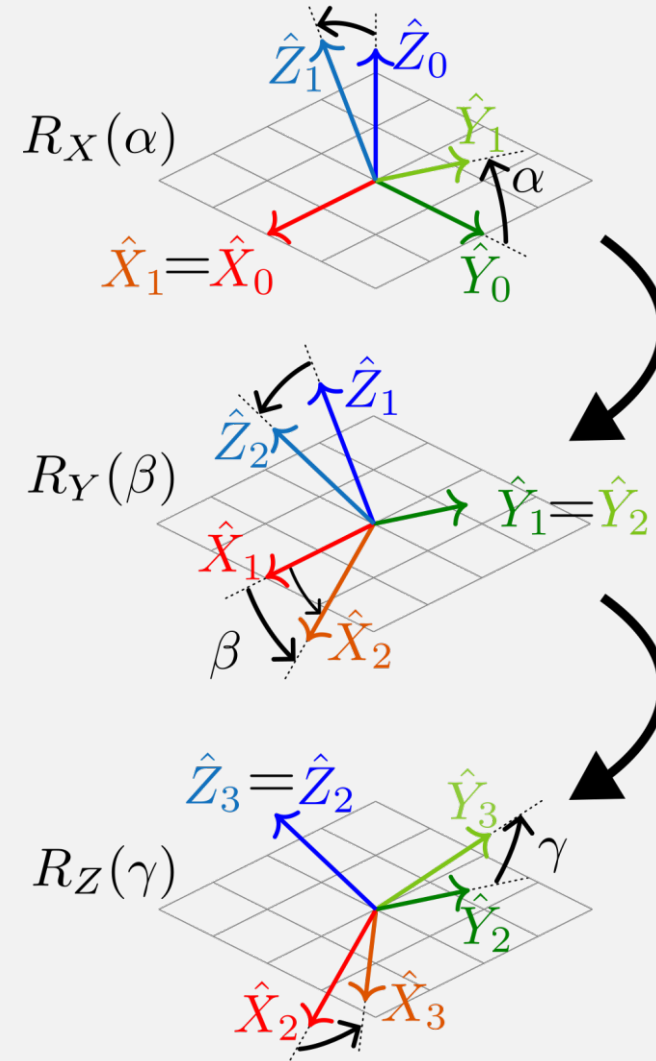


$$R_z(\theta) = \begin{bmatrix} c_\theta & -s_\theta & 0 \\ s_\theta & c_\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

ÂNGULOS DE EULER

- Podemos compor 3 rotações sequencialmente para obter uma rotação 3D
 - A sequência de rotações deve ser tal que duas rotações sucessivas não ocorram em torno do mesmo eixo
- Quando a primeira e terceira rotações são feitas em torno do mesmo eixo temos a parametrização de ângulos de Euler
 - Do contrário, temos os ângulos de Tait-Bryan
 - Caso específico: rotação $x \rightarrow y' \rightarrow z'' \Rightarrow \text{Roll}(x) - \text{Pitch}(y) - \text{Yaw}(z)$

$$\begin{aligned}
 RPY(\alpha, \beta, \gamma) &= R_z(\gamma)R_y(\beta)R_x(\alpha) \\
 &= \begin{bmatrix} c_\gamma & -s_\gamma & 0 \\ s_\gamma & c_\gamma & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} c_\beta & 0 & s_\beta \\ 0 & 1 & 0 \\ -s_\beta & 0 & c_\beta \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & c_\alpha & -s_\alpha \\ 0 & s_\alpha & c_\alpha \end{bmatrix} \\
 &= \begin{bmatrix} c_\beta c_\gamma & c_\gamma s_\beta s_\alpha - c_\alpha s_\gamma & s_\alpha s_\gamma + c_\alpha s_\beta c_\gamma \\ c_\beta s_\gamma & c_\alpha c_\gamma + s_\alpha s_\beta s_\gamma & c_\alpha s_\beta s_\gamma - c_\gamma s_\alpha \\ -s_\beta & c_\beta s_\alpha & c_\alpha c_\beta \end{bmatrix}
 \end{aligned}$$



ÂNGULOS DE EULER

- É possível extrair 3 ângulos de uma sequência qualquer dado uma matriz de rotação

- Ex: Roll-Pitch-Yaw

$$\begin{bmatrix} \alpha \\ \beta \\ \gamma \end{bmatrix} = \begin{bmatrix} \text{atan2}(c_{21}, c_{11}) \\ \text{atan2}\left(-c_{31}, \sqrt{c_{32}^2 + c_{33}^2}\right) \\ \text{atan2}(c_{32}, c_{33}) \end{bmatrix}$$

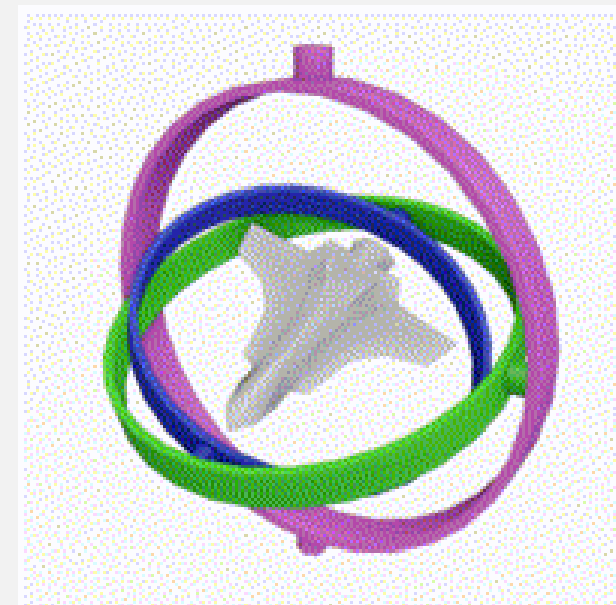
$\text{atan2}(y, x)$ é a função arco-tangente de 2 argumentos:

$$\text{atan2}(y, x) = \begin{cases} \text{atan}\frac{y}{x}, & x > 0 \\ \text{atan}\frac{y}{x} + \pi, & x < 0 \text{ e } y \geq 0 \\ \text{atan}\frac{y}{x} - \pi, & x < 0 \text{ e } y < 0 \end{cases}$$

- Um dos problemas de usar ângulos de Euler para representar rotações são as singularidades quando dois eixos coincidem (também chamado de *gimbal-locking*)

- Soluções possíveis:

- Trocar de sequência de ângulos de Euler perto da singularidade
- Usar outra parametrização sem singularidades (ex.: quaternions)



ROTAÇÕES EIXO-ÂNGULO

- Toda rotação também pode ser representada por só uma rotação de ângulo $\|\omega\| = \theta$ (magnitude) em torno de um eixo $\hat{\omega}$ (direção de rotação) – isso é, um vetor $\omega \in \mathbb{R}^3$

- Note que a rotação continua não sendo comutativa:

$$\omega_1 + \omega_2 \neq \omega_2 + \omega_1$$

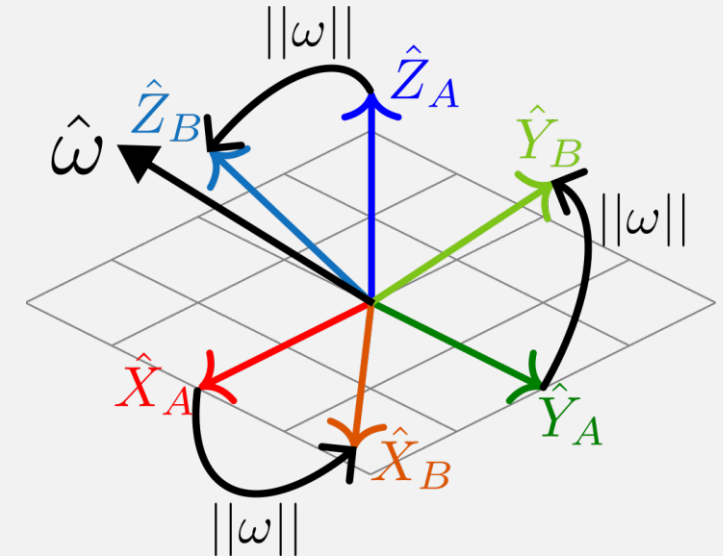
- Com parâmetros de rotação eixo-ângulo, a matriz de rotação é

$${}^A R_B(\omega) =$$

$$\begin{bmatrix} \omega_x^2(1 - c_\theta) + c_\theta & \omega_x\omega_y(1 - c_\theta) - \omega_z s_\theta & \omega_x\omega_z(1 - c_\theta) + \omega_y s_\theta \\ \omega_x\omega_y(1 - c_\theta) + \omega_z s_\theta & \omega_y^2(1 - c_\theta) + c_\theta & \omega_y\omega_z(1 - c_\theta) - \omega_x s_\theta \\ \omega_x\omega_z(1 - c_\theta) - \omega_y s_\theta & \omega_y\omega_z(1 - c_\theta) + \omega_x s_\theta & \omega_z^2(1 - c_\theta) + c_\theta \end{bmatrix}$$

- Dado uma matriz de rotação, os parâmetros eixo-ângulo são

$$\theta = \cos^{-1} \left(\frac{c_{11} + c_{22} + c_{33} - 1}{2} \right) \quad \hat{\omega} = \frac{1}{2 \sin \theta} \begin{bmatrix} c_{32} - c_{23} \\ c_{13} - c_{31} \\ c_{21} - c_{12} \end{bmatrix}$$



Eixo-ângulo contém 2 singularidades:

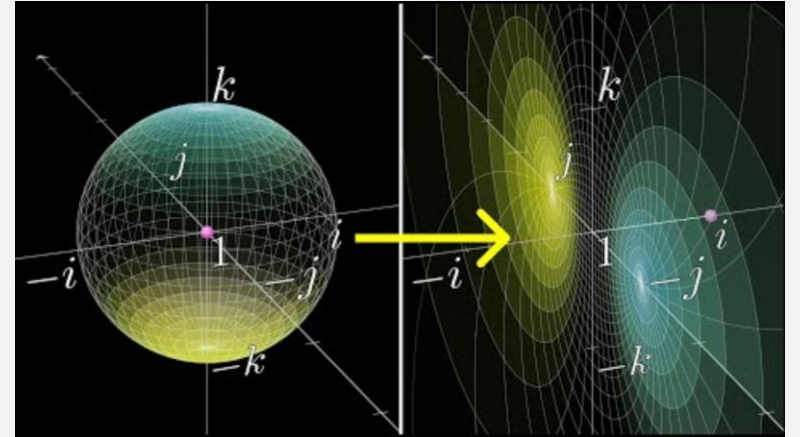
$\theta = 0 \rightarrow$ qualquer direção possível para o vetor

$\theta = \pi \rightarrow$ vetor pode apontar em duas direções opostas

QUATERNIONS UNITÁRIOS

- Quaternions unitários são uma parametrização de rotações com 4 parâmetros
- São “um escalar + um vetor”: $\mathbf{q} = (q_w, \overbrace{[q_x, q_y, q_z]}^{\text{vetor } \vec{q}})$
- O quaternion que representa nenhuma rotação é $\vec{q} = \mathbf{0}, q_w = 1$
- O inverso de um quaternion $\mathbf{q}$ é $\mathbf{q}^{-1} = (q_w, -[q_x, q_y, q_z])$
- Devem satisfazer a restrição $q_w^2 + q_x^2 + q_y^2 + q_z^2 = 1$
- Se tivermos um vetor de rotação $\hat{\omega} = (\theta, \hat{\omega})$, o quaternion correspondente é

$$\mathbf{q}(\hat{\omega}) = \left(\cos \frac{\theta}{2}, \sin \frac{\theta}{2} [\hat{\omega}_x, \hat{\omega}_y, \hat{\omega}_z] \right)$$



Quaternions são *notoriamente* difíceis de visualizar...

Material extra recomendado para os interessados: vídeos interativos de 3blue1brown e Ben Eater

QUATERNIONS UNITÁRIOS

- É possível converter de quaternions para uma matriz de rotação:

$${}^A R_B(\mathbf{q}) = \begin{bmatrix} q_w^2 + q_x^2 - q_y^2 - q_z^2 & 2q_xq_y - 2q_wq_z & 2q_xq_z + 2q_wq_y \\ 2q_xq_y + 2q_wq_z & q_w^2 - q_x^2 + q_y^2 - q_z^2 & 2q_yq_z - 2q_wq_x \\ 2q_xq_z - 2q_wq_y & 2q_yq_z + 2q_wq_x & q_w^2 - q_x^2 - q_y^2 + q_z^2 \end{bmatrix}$$

- ... E vice-versa:

$$\mathbf{q} = \begin{bmatrix} q_w \\ q_x \\ q_y \\ q_z \end{bmatrix} = \frac{1}{2} \begin{bmatrix} \sqrt{r_{11} + r_{22} + r_{33} + 1} \\ \text{sgn}(r_{32} - r_{23})\sqrt{r_{11} - r_{22} - r_{33} + 1} \\ \text{sgn}(r_{13} - r_{31})\sqrt{r_{22} - r_{33} - r_{11} + 1} \\ \text{sgn}(r_{21} - r_{12})\sqrt{r_{33} - r_{11} - r_{22} + 1} \end{bmatrix}$$

- A principal vantagem dos quaternions é que eles não apresentam singularidades
 - ...mas evidentemente a principal desvantagem é não ter interpretação física fácil

EM RESUMO: DIFERENTES FORMAS DE REPRESENTAR ROTAÇÕES 3D

Representação	Número de parâmetros	Facilidade de uso*	Representação única?	Legível por humanos?	Apresenta singularidades?
Matrizes de Rotação	9	**	Sim	+ -	Não
Ângulos de Euler	3	***	Não	Sim	Sim
Representação eixo-ângulo	4(3)	**	Não	Sim	Sim
Quaternions	4(3)	*	Sim	Não	Não



É possível converter de qualquer representação para qualquer outra representação...
mas nem sempre isso é feito de uma forma “suave”!

*Evidentemente, o quão fácil de usar algo é para alguém é altamente subjetivo.

JUNTANDO TUDO: TRANSFORMAÇÕES HOMOGÊNEAS

- Podemos representar a pose de qualquer *frame* com relação a qualquer outro *frame* através de uma translação ${}^A\mathbf{t}_B$ e uma rotação ${}^A\mathbf{R}_B$; ou seja, por uma transformação ${}^A\mathbf{T}_B = ({}^A\mathbf{t}_B, {}^A\mathbf{R}_B)$

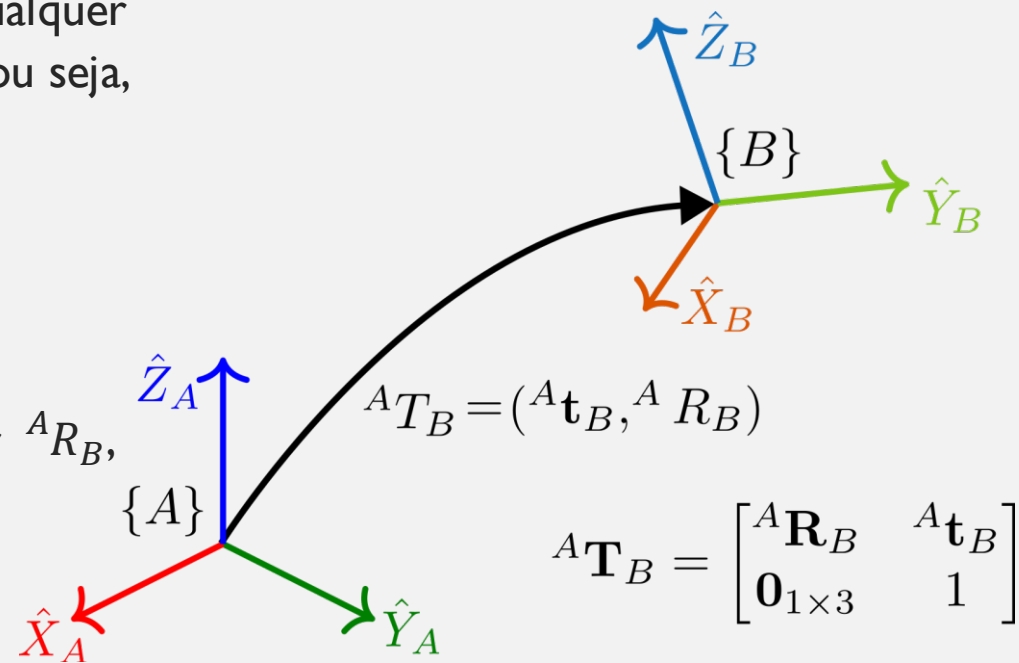
- Podemos juntar os dois se usarmos *coordenadas homogêneas*:

- Um ponto $\mathbf{p} = [p_x \ p_y \ p_z]^T$ é expresso em CH como $\mathbf{p} = [p_x \ p_y \ p_z \ 1]^T$

- Primeiro rotacionamos o frame $\{A\}$ em torno de sua origem por ${}^A\mathbf{R}_B$, depois movemos ela pelo vetor ${}^A\mathbf{t}_B$: ${}^A\mathbf{p} = {}^A\mathbf{R}_B {}^B\mathbf{p} + {}^A\mathbf{t}_B$

- Isso pode ser representado por uma matriz 4x4:

$${}^A\mathbf{T}_B = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} {}^A\mathbf{R}_B & {}^A\mathbf{t}_B \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix}$$



Logo, ${}^A\mathbf{p} = {}^A\mathbf{T}_B {}^B\mathbf{p}$ transforma a representação de $\mathbf{p}$ para o sistema $\{A\}$

TRANSFORMAÇÕES HOMOGÊNEAS

Propriedades:

- A inversa de uma matriz ${}^A T_B$ também é uma matriz de transformação, e tem a seguinte forma:

$$\begin{aligned} {}^A T_B &= \begin{bmatrix} {}^A R_B & {}^A t_B \\ 0_{1 \times 3} & 1 \end{bmatrix}^{-1} = \begin{bmatrix} {}^A R_B & -{}^A R_B^T {}^A t_B \\ 0_{1 \times 3} & 1 \end{bmatrix} \\ &= \begin{bmatrix} {}^A R_B^T & -{}^B R_A {}^A t_B \\ 0_{1 \times 3} & 1 \end{bmatrix} = {}^B T_A \end{aligned}$$

- Podemos compor transformações homogêneas:

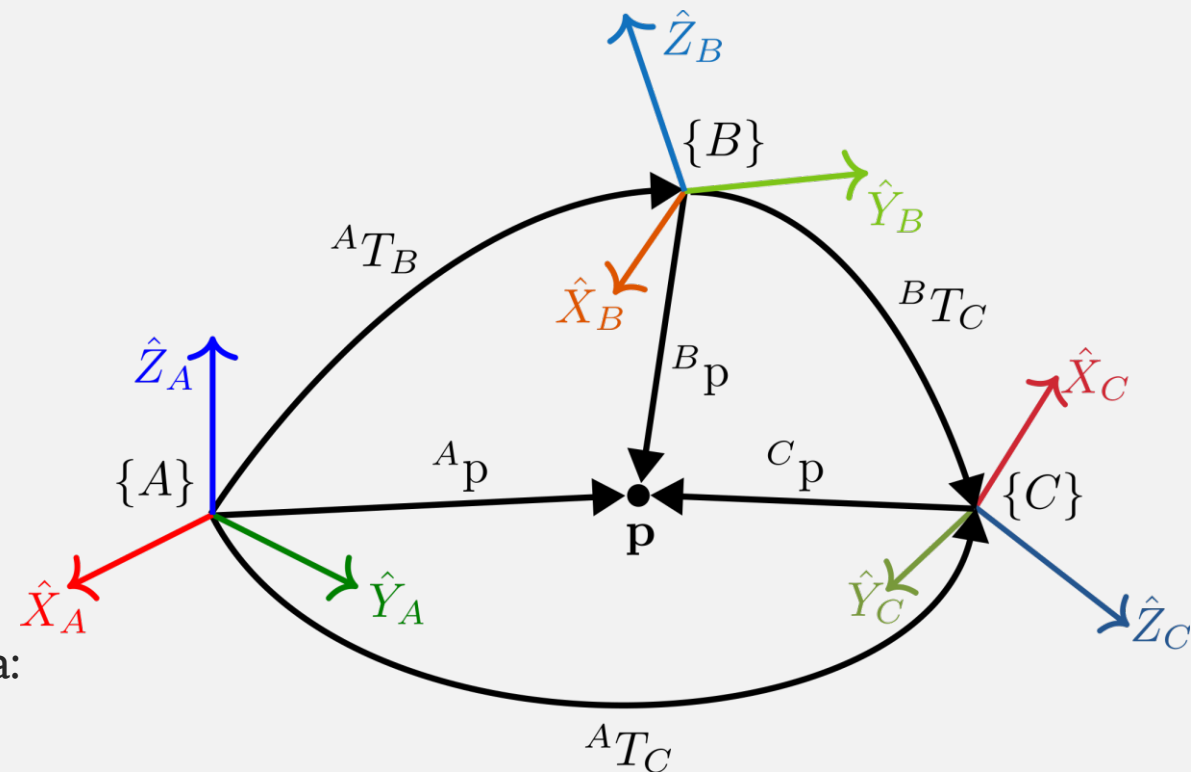
$${}^A T_C = {}^A T_B {}^B T_C$$

- A multiplicação de matrizes de transformação é associativa:

$$(T_1 T_2) T_3 = T_1 (T_2 T_3)$$

- ... mas não é comutativa:

$$T_1 T_2 \neq T_2 T_1$$



TRANSFORMAÇÕES HOMOGÊNEAS

Rotação e translação são casos particulares de transformações homogêneas

- O operador de rotação é apenas a parte de rotação da TH:

$${}^A\text{Rot}_B(\hat{\omega}, \theta) = \begin{bmatrix} {}^A R_B & 0_{3 \times 1} \\ 0_{1 \times 3} & 1 \end{bmatrix} =$$

- E similarmente, translação é a parte de translação, com rotação identidade:

$${}^A\text{Trans}_B(\mathbf{t}) = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- Note que ${}^A T_B = {}^A\text{Trans}_B {}^A\text{Rot}_B$
- O uso de rotações e transformações sequenciais é usado nos parâmetros de Denavit-Hartenberg para manipuladores

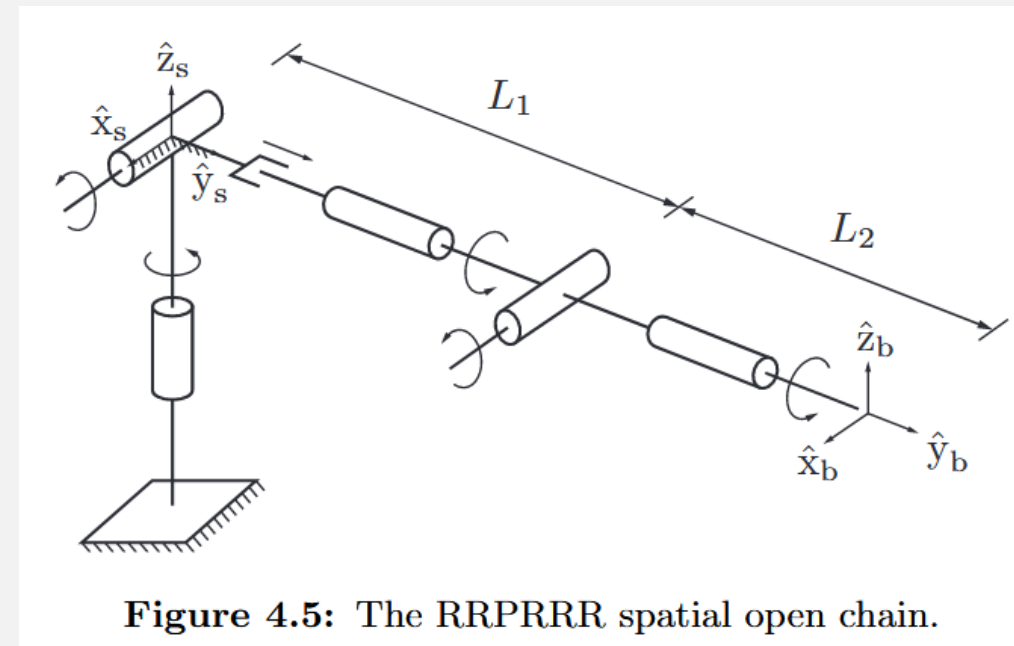
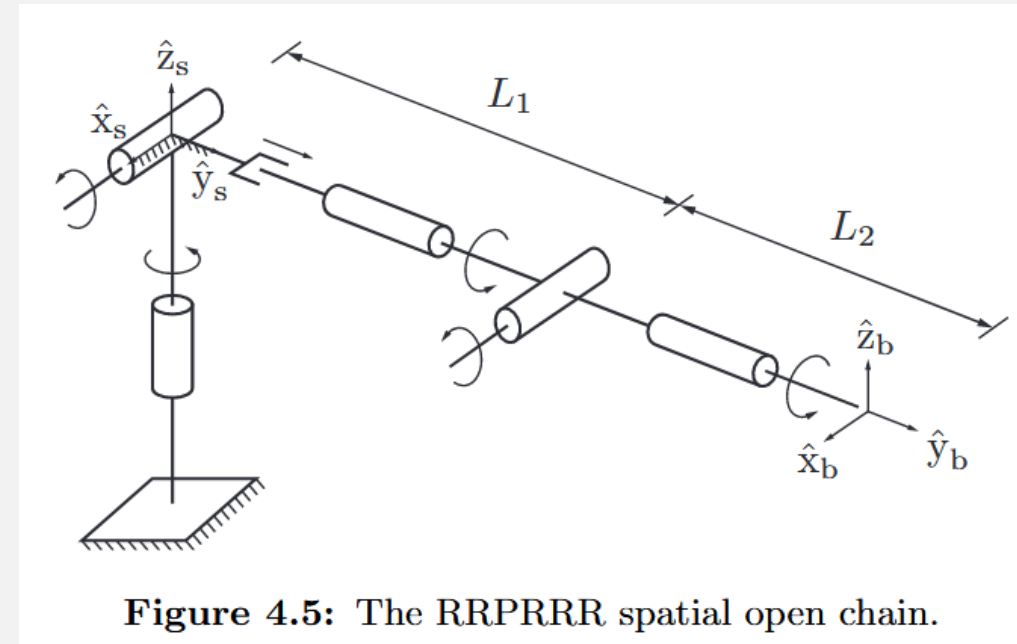


Figure 4.5: The RRPRRR spatial open chain.

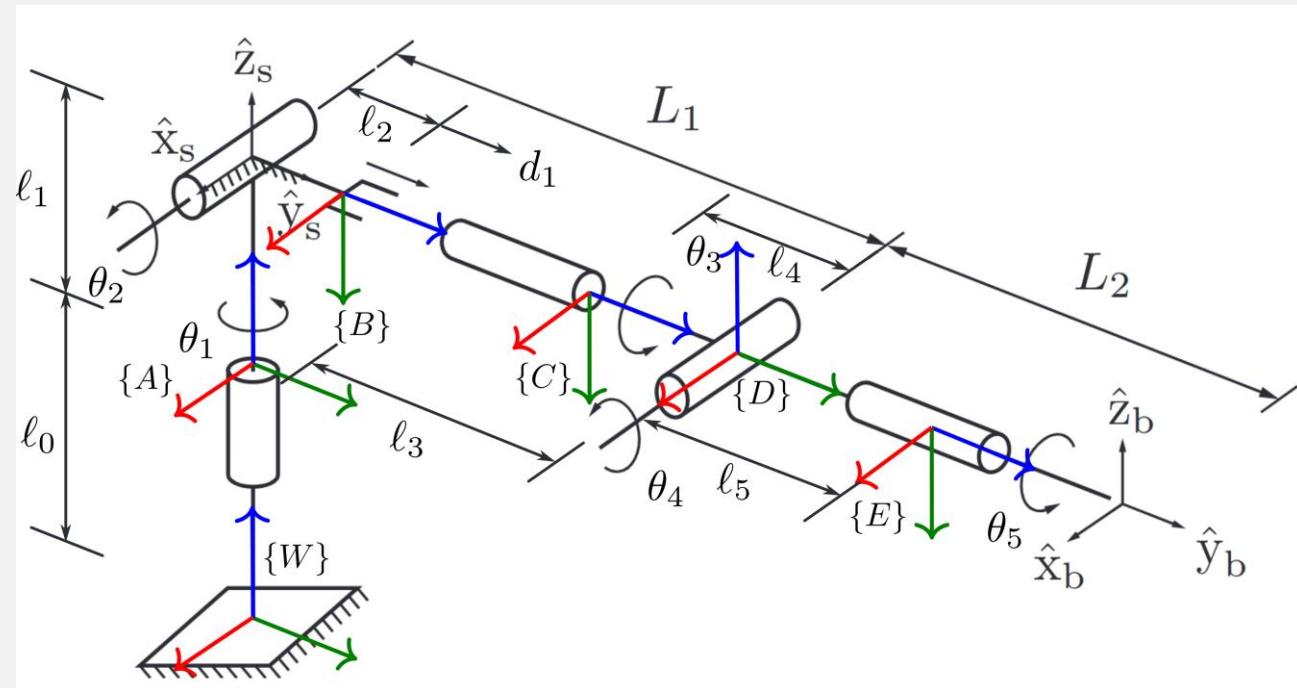
TRANSFORMAÇÕES HOMOGÊNEAS

- Ex: Considere o robô diagramado ao lado. Como ficaria a transformação do frame $\{b\}$, do punho, com relação ao frame $\{s\}$?
- Primeira coisa a notar: que tipos de juntas temos no robô?
 - Resposta: pelo diagrama, isso se trata de uma cadeia cinemática RRPRRR
 - Ou seja, 2x(revoluta) -> 1x(prismática) -> 3x(revoluta)



TRANSFORMAÇÕES HOMOGÊNEAS

- Ex: Considere o robô diagramado ao lado. Como ficaria a transformação do frame $\{b\}$, do punho, com relação ao frame $\{s\}$, *nessa posição*?
- Primeira coisa a notar: que tipos de juntas temos no robô?
 - Resposta: pelo diagrama, isso se trata de uma cadeia cinemática RRP₂RRR
 - Ou seja, 2x(revoluta) -> 1x(prismática) -> 3x(revoluta)
- Segunda coisa a notar: onde vamos colocar os frames de nossos elos?
 - Resposta: A variar - depende do fabricante do robô
 - Geralmente segue-se a convenção de Denavit-Hartenberg
 - Aqui decidimos fazer um pouco diferente – figura ao lado mostra os nossos frames



TRANSFORMAÇÕES HOMOGÊNEAS

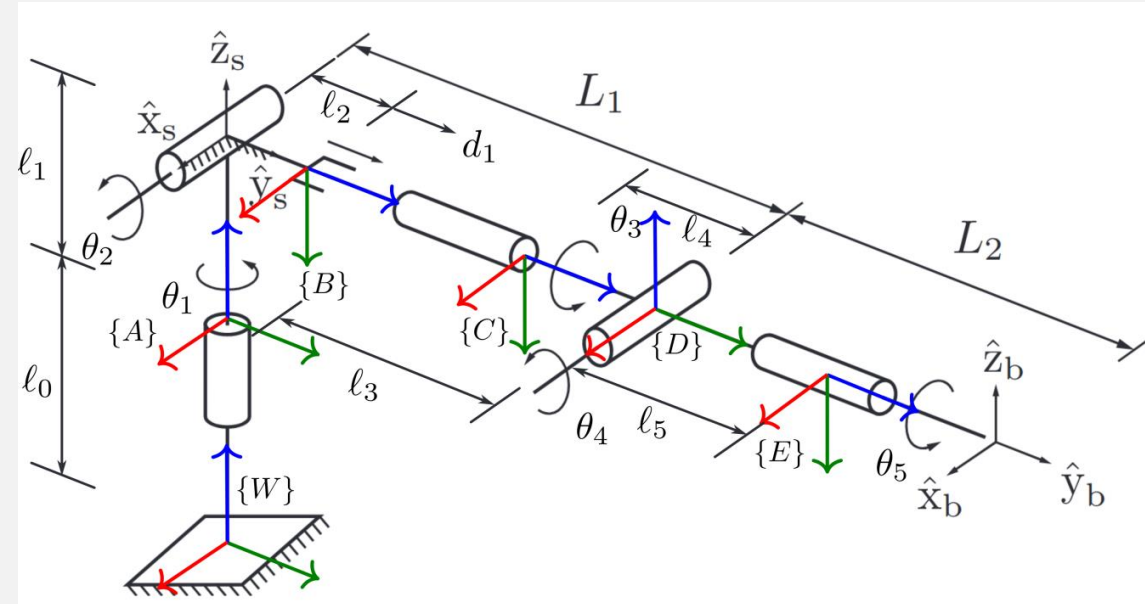
- Vamos por partes...

- $${}^S T_B = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & c_{\theta'_2} & -s_{\theta'_2} & \ell_2 \\ 0 & s_{\theta'_2} & c_{\theta'_2} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = t_y(\ell_2)R_X(\theta'_2), \theta'_2 = -90^\circ$$

- $${}^B T_C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \ell_3 \\ 0 & 0 & 0 & 1 \end{bmatrix} = t_z(\ell_3)$$

- $${}^C T_D = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & c_{\theta'_3} & -s_{\theta'_3} & 0 \\ 0 & s_{\theta'_3} & c_{\theta'_3} & \ell_4 \\ 0 & 0 & 0 & 1 \end{bmatrix} = t_z(\ell_4)R_X(\theta'_3), \theta'_3 = 90^\circ$$

- $${}^D T_E = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & c_{\theta'_4} & -s_{\theta'_4} & \ell_5 \\ 0 & s_{\theta'_4} & c_{\theta'_4} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = t_y(\ell_5)R_X(\theta'_4), \theta'_4 = -90^\circ$$

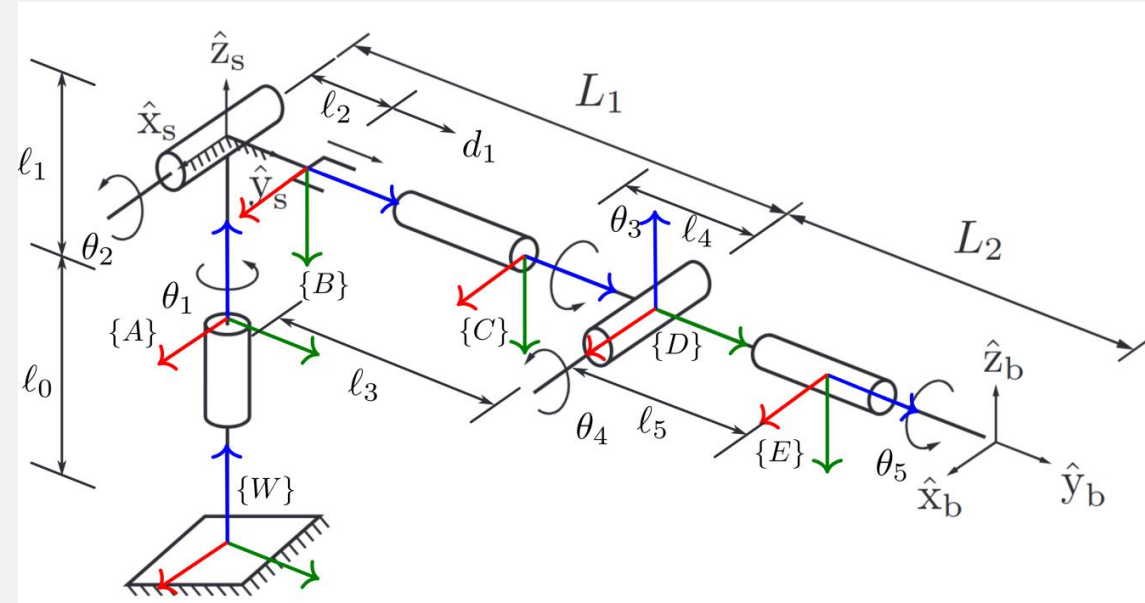


TRANSFORMAÇÕES HOMOGÊNEAS

$$\begin{aligned}
 {}^E T_b &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & c_{\theta'_5} & -s_{\theta'_5} & 0 \\ 0 & s_{\theta'_5} & c_{\theta'_5} & L_2 - \ell_5 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
 &= t_z(L_2 - \ell_5) R_X(\theta'_5), \theta'_5 = 90^\circ
 \end{aligned}$$

Agora, é “só” juntar tudo:

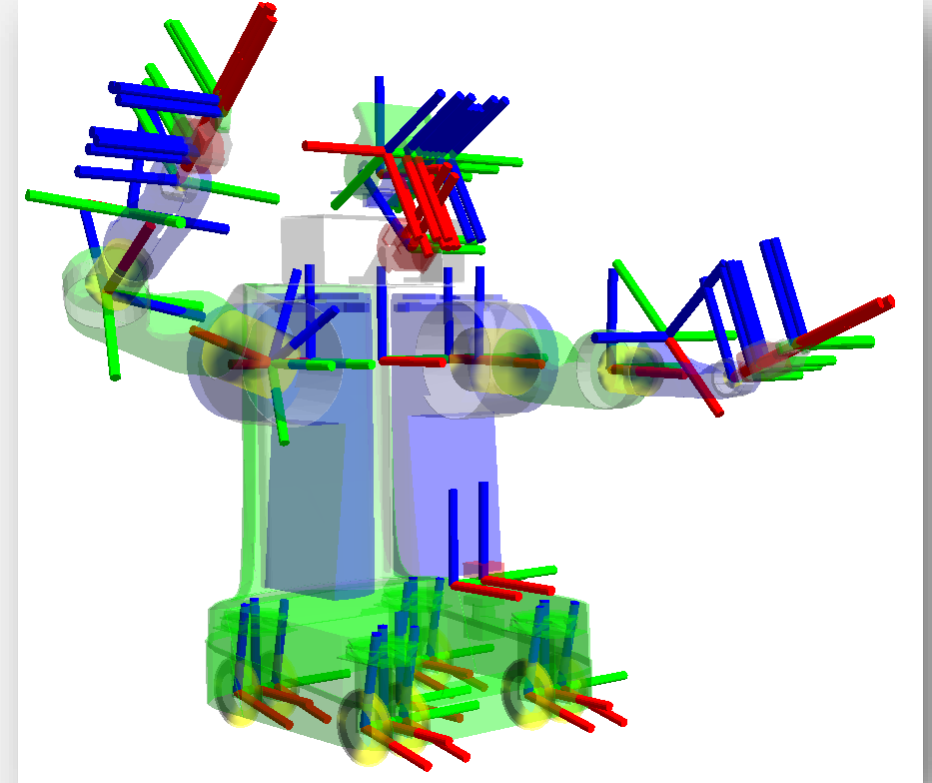
$$\begin{aligned}
 {}^S T_b &= {}^S T_B {}^B T_C {}^C T_D {}^D T_E {}^E T_b = \\
 &= t_y(\ell_2) R_X(\theta'_2) t_z(\ell_3) t_z(\ell_4) R_X(\theta'_3) t_y(\ell_5) R_X(\theta'_4) t_z(L_2 - \ell_5) R_X(\theta'_5)
 \end{aligned}$$



Bastante coisa. Felizmente, não temos que implementar todos esses cálculos no ROS 2!

O SUBSISTEMA TF: COORDENADAS NO ROS2

- ROS2 tem um subsistema dedicado inteiramente a lidar com *frames* e transformações: tf2
- Tf2 mantém as relações de transformação entre os vários frames em uma estrutura de árvore com *buffer*, disponível ao usuário
- Permite que o usuário busque relações entre quaisquer dois *frames* em qualquer instante de tempo
- Duas tarefas principais que qualquer nó pode fazer:
 - *Broadcasting* transformações ao sistema
 - Escutar por transformações relevantes



O SUBSISTEMA TF: COORDENADAS NO ROS2

- ROS2 implementa o sistema `tf2` com dois tópicos que recebem transformações (`/tf` e `/tf_static`) com mensagens do tipo `tf2_msgs/msg/TFMessage`
 - `/tf`: Transformações que variam dinamicamente (e.g.: juntas de um manipulador, pose de um drone). Por default, tem duração válida de 10 seg.
 - `/tf_static`: Transformações que *não* variam com o tempo (e.g.: geometria de um robô). Tem QoS `transient_local`: qualquer nó que se inscreve nesse tópico tem acesso a todas as transformações já publicadas.
- Todo frame deve ter um nome; alguns são especiais por convenção (`/world`, `/map`, `/odom`, `/base_link`)
- Publicação e inscrição de mensagens é um pouco diferente de só usar publishers e subscribers.

```
$ ros2 interface show tf2_msgs/msg/TFMessage

geometry_msgs/TransformStamped[] transforms
  std_msgs/Header header
    builtin_interfaces/Time stamp
      int32 sec
      uint32 nanosec
  string frame_id
  string child_frame_id
  Transform transform
    Vector3 translation
      float64 x
      float64 y
      float64 z
    Quaternion rotation
      float64 x 0
      float64 y 0
      float64 z 0
      float64 w 1
```

O SUBSISTEMA TF: COORDENADAS NO ROS2

- Transformações estáticas podem ser publicadas a partir de nós, linha de comando (CLI) ou arquivos de launch
 - Geralmente o recomendado é via launch ou CLI
 - Linha de comando: nó `static_transform_publisher` do pacote `tf2_ros`

```
$ ros2 run tf2_ros static_transform_publisher --x 0 --y 0 --z 1 --yaw 0 --pitch 0 --roll 0 --frame-id world --child-frame-id my_child_frame
```

- Arquivo de launch XML: launch com argumentos no nó `static_transform_publisher`

```
<launch>
  <node
    pkg="tf2_ros" exec="static_transform_publisher"
    args="--x 0 --y 0 --z 1 --yaw 0 --pitch 0 --roll 0 --frame-id world --child-frame-id my_child_frame"
  />
</launch>
```

O SUBSISTEMA TF: COORDENADAS NO ROS2

- Publicação: `tf2_ros::TransformBroadcaster`

```
// Dentro da classe do nó: cria o broadcaster e a mensagem
tf_broadcaster_ =
    std::make_unique<tf2_ros::TransformBroadcaster>(*this);
geometry_msgs::msg::TransformStamped tf_A_world;

// Cria uma mensagem e preenche com informação relevante
tf_A_world.header.stamp = this->get_clock()->now();
tf_A_world.header.frame_id = "world";
tf_A_world.child_frame_id = "A";
tf_A_world.transform.translation.x = 1.0; // coord X

// Rotação: Cria quaternion via Roll, Pitch Yaw
tf2::Quaternion q;
q.setRPY(0, 0, msg->theta);
tf_A_world.transform.rotation.x = q.x();
tf_A_world.transform.rotation.y = q.y();
tf_A_world.transform.rotation.z = q.z();
tf_A_world.transform.rotation.w = q.w();

tf_broadcaster_->sendTransform(t); // Publica transformada
```

Inscrição: `tf2_ros::Buffer` e
`tf2_ros::TransformListener`

```
// Dentro da classe do nó: cria o buffer e o listener
tf_buffer_ =
    std::make_unique<tf2_ros::Buffer>(this->get_clock());
tf_listener_ =
    std::make_shared<tf2_ros::TransformListener>(*tf_buffer_);

//...

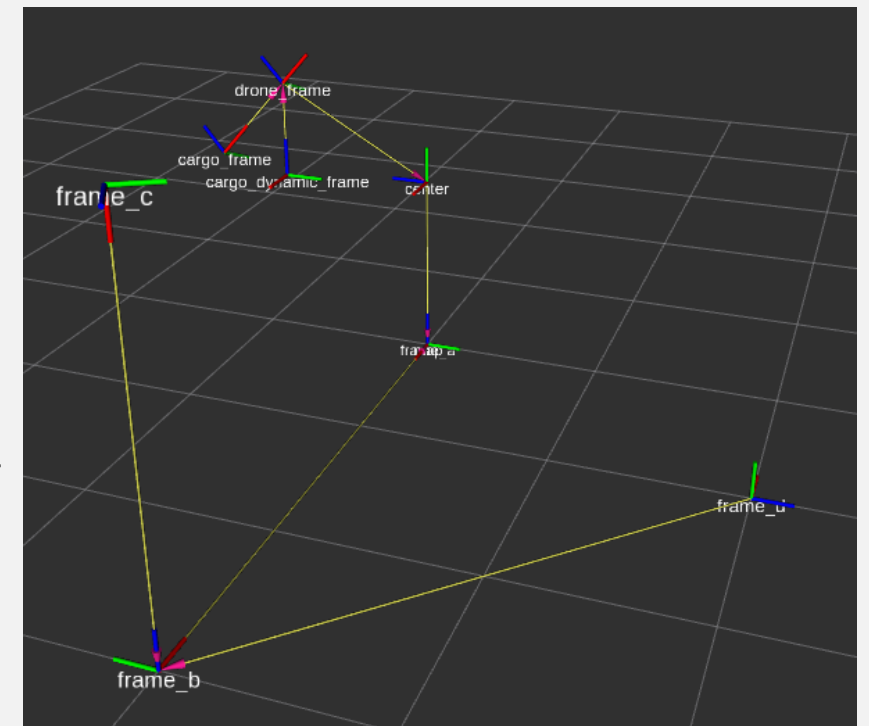
geometry_msgs::msg::TransformStamped odom2obstacle;
odom2obstacle = tf_buffer_->lookupTransform(
    "odom", //Para a frame odom
    "obstacle", // Da frame obstacle
    tf2::TimePointZero); // Solicita o último frame disponível
```

O SUBSISTEMA TF: COORDENADAS NO ROS2

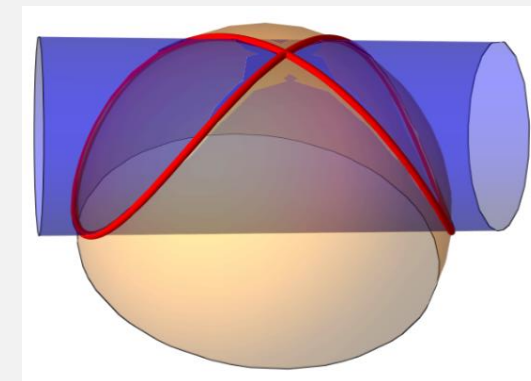
- Vamos dar uma olhada em um pacote que publica periodicamente uma curva de Viviani no espaço (imagine um drone voando em uma Figura 8 tridimensional)
- Faça o download do pacote tf2_demo, na pasta Aula4 no repositório https://github.com/nycolasRamires/ros2_ufc/
- Coloque a pasta tf2_demo no diretório src do seu workspace
- Rode os comandos

```
colcon build --symlink-install && source
install/setup.bash
```

no diretório raiz do workspace
- Lance o pacote com
 - `ros2 launch tf2_demo demo_bringup.launch.xml config:=True`



Vista do Rviz do pacote completo



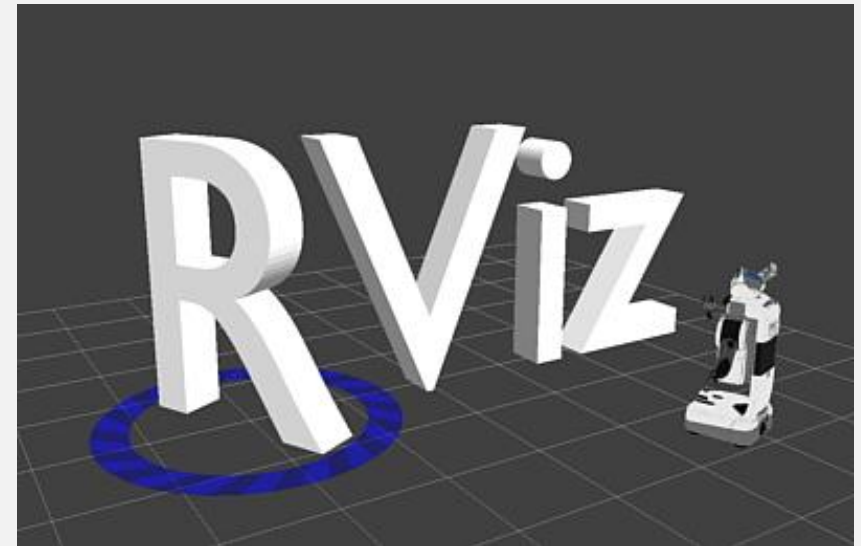
Curva de Viviani (em vermelho)

RVIZ: FERRAMENTA DE VISUALIZAÇÃO 3D DO ROS

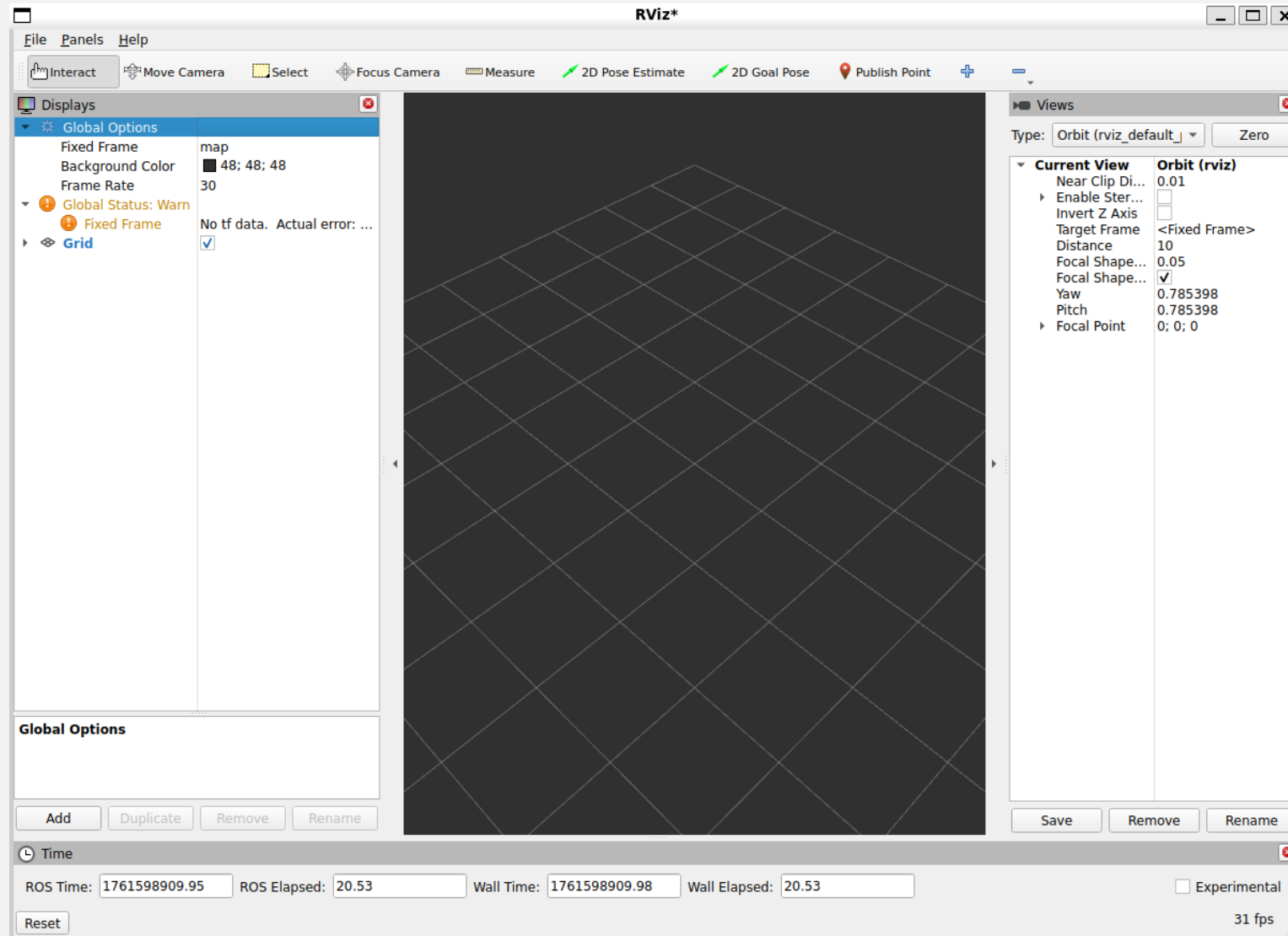
- O RViz(2) é a ferramenta padrão de visualização do ROS(2)
- Fornece integração direta com subsistema do ROS e mais uma hoste de plugins
- Customizável – podemos escrever nossos próprios plugins/dashboards para Rviz
- Pode ser invocado por linha de comando:

```
ros2 run rviz2 rviz2
```

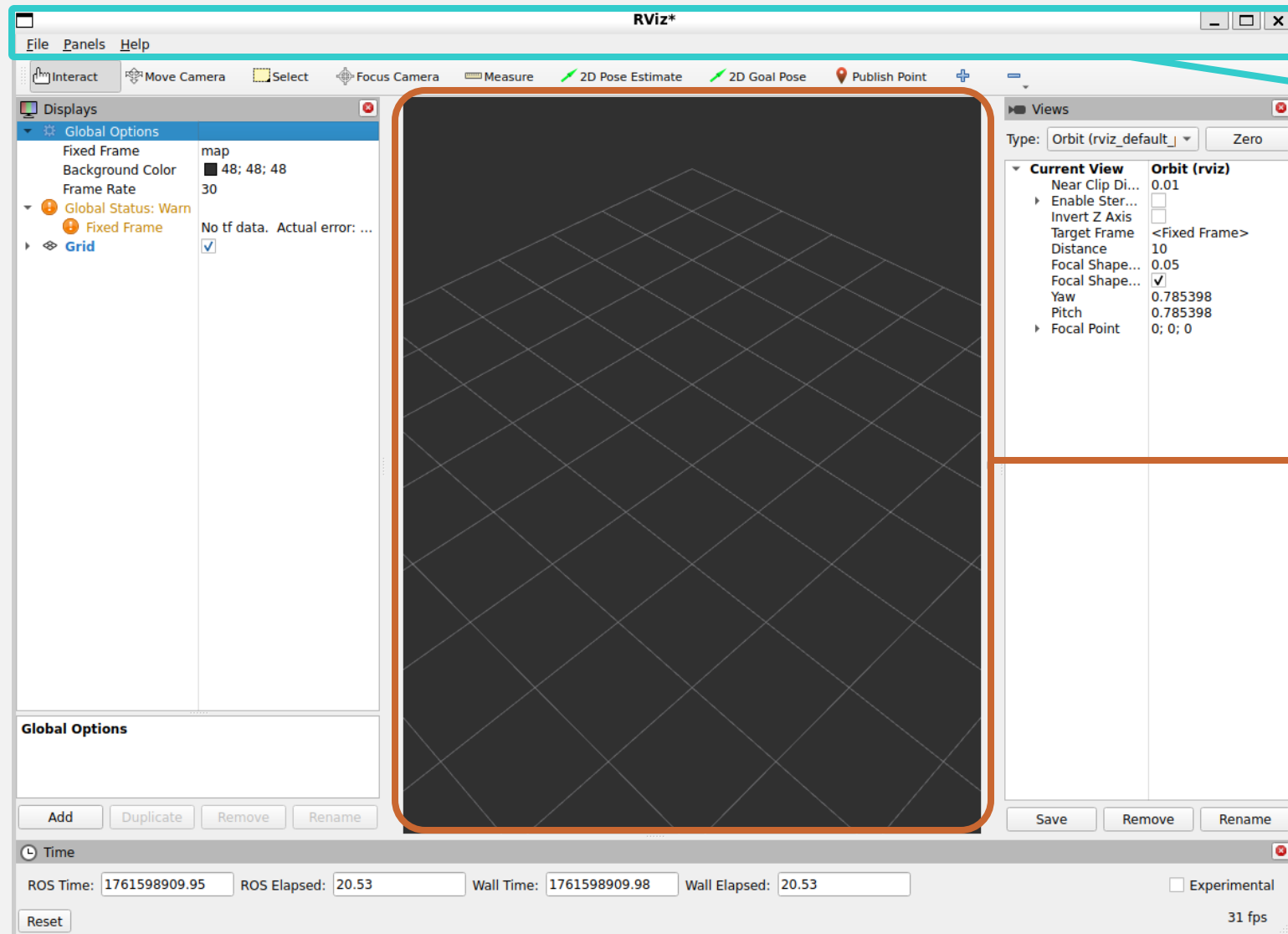
- Possui uma suíte de marcadores para visualização e *debug*
- Podemos criar e salvar nossos próprios arquivos de configuração para uso posterior
- Dependendo do framework pode ser usado para interagir graficamente com os robôs (ex: nav2, Moveit!)



INTERFACE RVIZ



INTERFACE RVIZ



Barra de opções,
ferramentas, ajuda,
ações da janela, etc...

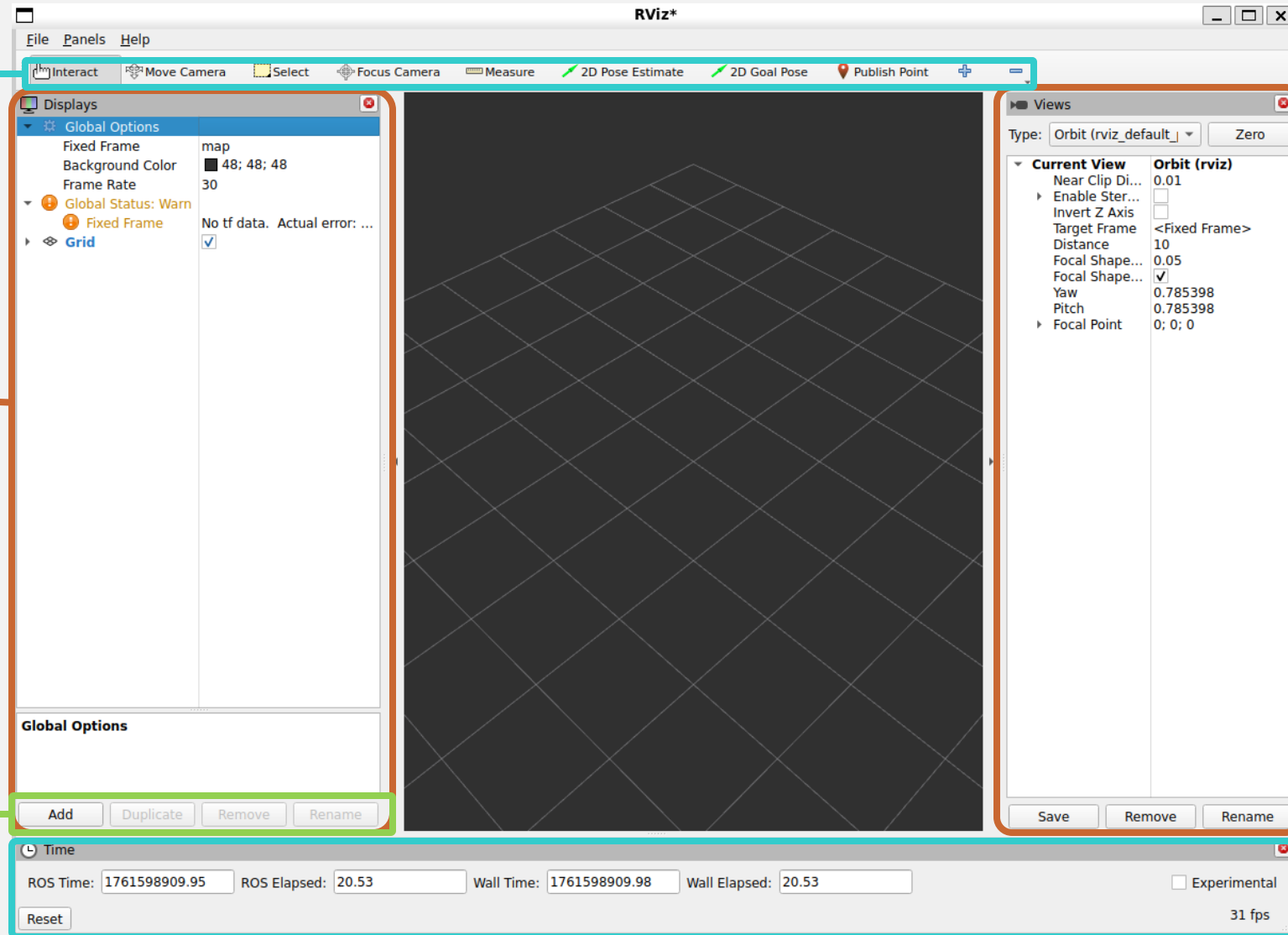
Vista em 3D do que
queremos visualizar

INTERFACE RVIZ

Painel de ferramentas –
utilitários para
interagir com a
visualização

Lista de displays -
Configuração e
adição de novos
displays para
visualização

Adição, duplicação,
remoção de
objetos de display

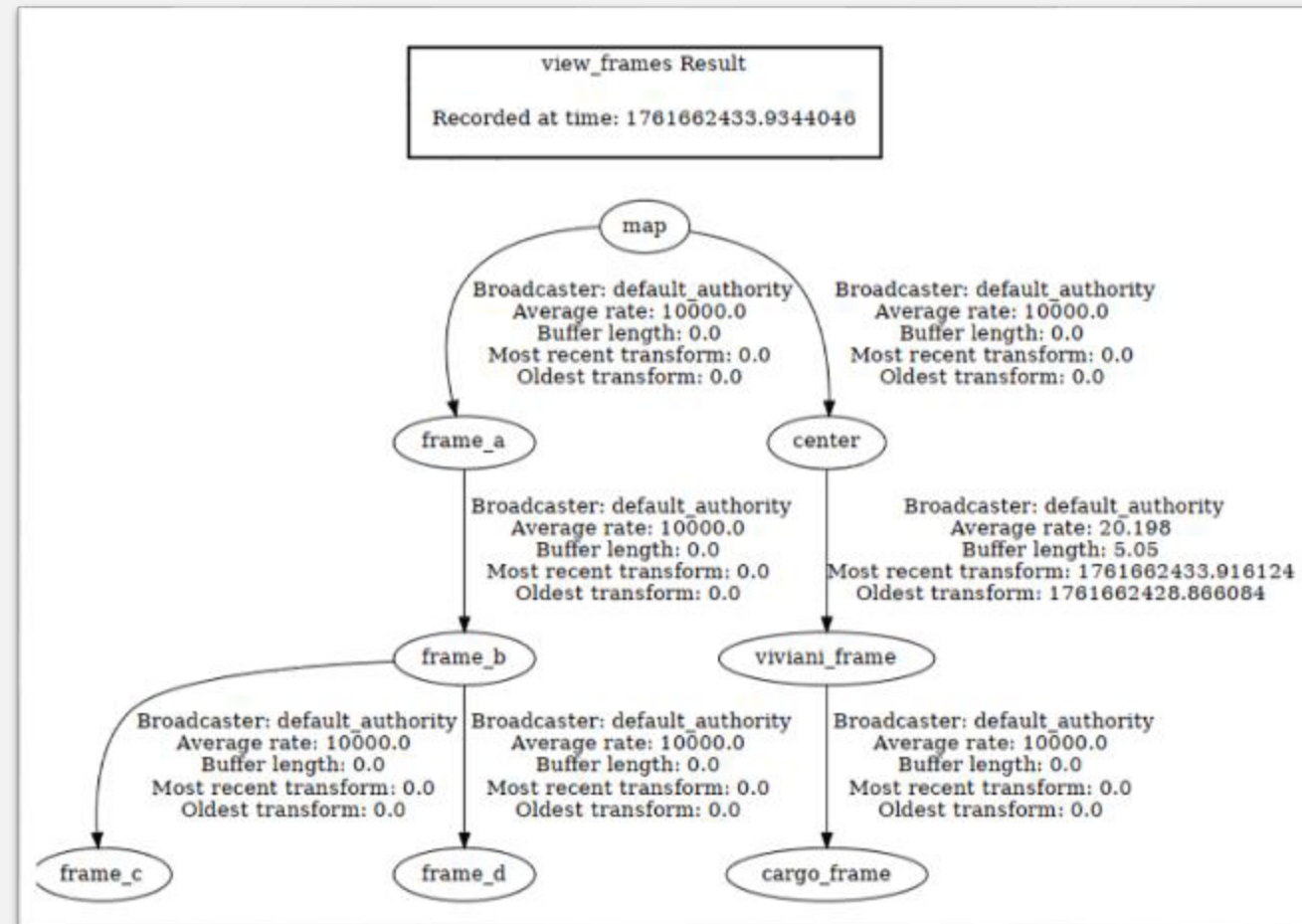


Painel de vistas –
permite customizar
movimento da
câmera, criar e
salvar vistas pre-
definidas

Painel de tempo –
compara tempo
real (*wall time*) VS
tempo no ROS. Útil
com simulações.

COMO VISUALIZAR TRANSFORMAÇÕES NO ROS2?

- O sistema tf2 vem com algumas ferramentas para debugar transformações. Vamos olhar algumas delas:
- `tf_tree`:
 - Escaneia todas as transformações disponíveis atualmente
 - Constroi a árvore de transformações e printa ela em um arquivo .pdf para visualização
 - Invocado via linha de comando:
`ros2 run tf2_tools view_frames`



COMO VISUALIZAR TRANSFORMAÇÕES NO ROS2?

- **tf2_echo**
 - Escuta e republica transformações entre dois frames quaisquer (se existir)
 - Chamada: `ros2 run tf2_ros tf2_echo [source_frame] [target_frame] [--echo_rate]`

```
At time 0.0
- Translation: [0.000, 0.000, 2.000]
- Rotation: in Quaternion [0.000, 0.000, 0.000, 1.000]
- Rotation: in RPY (radian) [0.000, -0.000, 0.000]
- Rotation: in RPY (degree) [0.000, -0.000, 0.000]
- Matrix:
  1.000  0.000  0.000  0.000
  0.000  1.000  0.000  0.000
  0.000  0.000  1.000  2.000
  0.000  0.000  0.000  1.000
```

Exemplo de saída do tf2_echo

COMO VISUALIZAR TRANSFORMAÇÕES NO ROS2?

- **tf2_monitor**

- Mostra estatísticas da transformação entre quaisquer dois frames
- Uso: `tf2_monitor [frame_a] [frame_b]`

```
Gathering data on frame_a -> frame_b for 10 seconds...
```

```
RESULTS: for frame_a to frame_b
```

```
Chain is: frame_a -> frame_b
```

```
Net delay      avg = 3.35195e+08: max = 1.76166e+09
```

```
Frames:
```

```
Frame: frame_a, published by <no authority available>, Average Delay: 48.3419, Max Delay: 48.3419
```

```
Frame: frame_b, published by <no authority available>, Average Delay: 48.3326, Max Delay: 48.3326
```

```
All Broadcasters:
```

```
Node: <no authority available> 20.6574 Hz, Average Delay: 1.34906 Max Delay: 48.3635
```

Exemplo de saída do `tf2_monitor`

(O grande delay é devido a ambos os frames serem estáticos)